Software Requirement Specification Document

1 . Introduction

This document specifies the functional and non-functional requirements for the "University Academic Content Management System". It clearly define the system's expected behavior, features, constraints, and performance requirements before development begins

1.1 Scope

WHAT the system will do at a high level.

- Support **multiple universities** within a single platform.
- Allow storage and management of **previous year question papers, notes, and paper solutions**.
- Implement **role-based access control**, where different users (university representatives, department representatives, year representatives, instructors, and students) have different permissions.
- Provide a **structured hierarchy** (University → Department → Program → Year).
- Include a **ranking or contribution system** to recognize active contributors.
- Allow students to securely access and download academic content.

1.2 Definitions

- **SRS (Software Requirements Specification):** A document that describes system requirements in detail.

- **RBAC (Role-Based Access Control):** A method of restricting system access based on user roles.
- **JWT (JSON Web Token):** A secure token used for user authentication and authorization.
- **CDN (Content Delivery Network):** A network used to deliver content quickly to users.
- **API (Application Programming Interface):** A set of rules that allows different software components to communicate.

1.3 References

This SRS is prepared with reference to the following standards:

- **IEEE ANSI 830-1984** – IEEE Recommended Practice for Software Requirements Specifications.
- **IEEE ANSI 830-1998** – IEEE Recommended Practice for Software Requirements Specifications (Updated Version).

1.4 Stakeholder

- Global Administrator
- University Representatives
- Department Representatives
- Program Representatives
- Session Representatives
- Students (Users)
- Developers
- Instructor (as client/customer)

02 . Functional Requirements

1 . Global Admin Responsibility

1.Add multiple universities in the application
2.Make a university representative per university
3.can contact to any user ⇒ user,university admins

2. University Representative Responsibility

```
-> Add all the depertments of the university
-> Assign department representative per department
-> Add the university exam formate
```

3. Department Representative Responsibility

```
-> Department Rep. will add all the programs of the departmen
t
-> will add duration of the program
-> Department Rep will make program Representative
-> Assign a program representative per Program
```

4 . Program Representative Responsitivity

```
   -> Makes Per year Representative (it will be a session Rep
resentative)
```

5. Session (Year) Representative Responsibilities

```
-> Add All subjects of the sem
-> Verify all the paper
-> upload all the papers
```

6. User

```
User can register According to own university
user can see all the papers of that university according to f
ilters
user can add papers that will be verified by admin
user can add notes , paper solution
user will get points according to unloaded content
all users will be shown according to rank in a university
```

03 . Non Functional Requirements

3.1 Performance

3.1.1 Response Time

The system shall meet the following performance benchmarks under normal operating conditions:

- Page load time shall not exceed **3 seconds** for standard user requests.
- API response time shall be **less than 2 seconds** for 95% of requests.
- Upload initialization shall respond within **1 second**.
- Search and filtering results shall be returned within **2 seconds**.
- Authentication and login process shall complete within **2 seconds**.

How This Will Be Achieved

- **Spring Boot microservices architecture** to distribute workload.
- **Kubernetes auto-scaling (HPA)** to scale pods based on CPU/memory usage.
- **Kafka** for asynchronous processing (e.g., notifications, logging, ranking updates).
- **Database indexing** on frequently queried columns (university_id, program_id, subject_id).

- **Redis caching** for frequently accessed data (subjects list, exam patterns).
- **CDN or object storage (MinIO)** for file serving to reduce server load.
- **Pagination and lazy loading** to avoid loading large datasets at once.

3.1.2 Throughput

The system shall support:

- At least **1000 concurrent active users**.
- At least **200 file uploads per hour**.
- At least 2**000 API requests per minute** under peak load.

How This Will Be Achieved

- **Horizontal scaling using Kubernetes pods.**
- **Docker containerization** to ensure consistent deployments.
- **Load balancing via Kubernetes services or Ingress controller.**
- **Kafka event streaming** to decouple heavy operations.
- Optimized SQL queries and connection pooling (HikariCP in Spring Boot).

3.2 Security Considerations

The system shall ensure secure access and data protection through the following mechanisms:

- The system shall implement **Role-Based Access Control (RBAC)**.
- The system shall use **JWT-based authentication and authorization**.
- All communication between client and server shall use **HTTPS (TLS encryption)**.
- File upload shall allow only **PDF and image formats (JPEG, PNG)**.

- Maximum file size shall be limited to **20 MB per file**.
- The system shall implement **rate limiting** to prevent abuse.
- Security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) shall be enabled.
- Passwords shall be stored using **BCrypt hashing**.
- Microservices communication shall be secured using internal network policies.

How This Will Be Achieved

- Spring Security with JWT filters.
- API Gateway for centralized authentication.
- Kubernetes secrets for secure credential management.
- Input validation and file type validation at backend.
- Rate limiting via API Gateway or Redis-based throttling.

3.3 Usability

The system shall provide a user-friendly interface with the following characteristics:

- The interface shall be **web-based**.
- The system shall provide **clear and structured navigation**.
- Upload process shall display a **real-time progress indicator**.
- Filtering and search shall be intuitive and easy to use.
- Error messages shall be clear, descriptive, and user-friendly.
- Forms shall include validation feedback before submission.

How This Will Be Achieved

- Responsive frontend framework (React/Next.js or similar).

- RESTful API integration.
- Proper UX design with structured layout.
- Client-side validation combined with backend validation.
- Loading indicators and progress bars for uploads.

3.4 Reliability and Availability

The system shall ensure stable operation and fault tolerance.

- The system shall maintain **99% availability annually**.
- Services shall operate independently within a microservices architecture.
- Failure of one service shall not crash the entire system.
- The system shall support automated data backup.
- The system shall support automatic service restart in case of failure.
- Logs and monitoring shall be enabled for failure tracking.

How This Will Be Achieved

- Kubernetes self-healing (auto-restart failed pods).
- Microservices isolation.
- Health checks (liveness and readiness probes).
- Centralized logging (ELK stack or similar).
- Periodic database backup strategy.
- Kafka for reliable event handling.

3.5 Format (Website)

04 . Time Frame and Criteria

4.1 Project Schedule

The project development shall follow the timeline below:

Phase 1: Requirement Analysis

Duration: 2 Weeks

Start Date: 11 February 2026

End Date: 24 February 2026

Phase 2: System Design

Duration: 3 Weeks

Start Date: 25 February 2026

End Date: 17 March 2026

Activities:

- Architecture design (Microservices)
- Database schema design
- API design
- UI wireframes
- LLD and class diagrams

Phase 3: Development

Duration: 5 Weeks

Start Date: 18 March 2026

End Date: 22 April 2026

Activities:

- Backend service implementation
- Frontend development

- Role-based authentication
- Content upload and ranking logic

Phase 4: Testing

Duration: 2 Weeks

Start Date: 22 April 2026

End Date: 02 May 2026

Activities:

- Unit testing
- Integration testing
- Performance testing
- Security validation

Phase 5: Documentation and Final Submission

Duration: 1 Week

Start Date: 02 May 2026

End Date: 07 May 2026

Activities:

- Final documentation
- Report formatting
- Final review
- Submission preparation

05. Acceptance Criteria

The project shall be considered complete and acceptable when the following conditions are satisfied:

5.1 Functional Completion

- All functional requirements specified in this SRS document shall be fully implemented.
- All modules including authentication, role management, university management, content upload, filtering, and download shall function as defined.
- Users shall be able to register, login, and access features according to their assigned roles.
- Authorized users shall be able to upload academic content successfully.
- Users shall be able to download available content without errors.

5.2 Role-Based Access Validation

- Role-Based Access Control (RBAC) shall correctly restrict access to protected resources.
- Unauthorized users shall not be able to access restricted endpoints.
- Each role (Admin, University Representative, Department Representative, Year Representative, Student/User) shall only access permitted functionalities.
- JWT authentication shall successfully validate and authorize requests.

5.3 Performance Validation

- Page load time shall not exceed 3 seconds under normal load.
- API response time shall be less than 2 seconds for most requests.
- The system shall support at least 1000 concurrent users without failure.
- File uploads and downloads shall function smoothly under expected traffic conditions.

Performance shall be verified through load testing and performance testing tools.

5.4 Security Validation

- All communication shall use HTTPS.
- Passwords shall be securely encrypted.
- File upload restrictions (format and size limits) shall be enforced.
- Rate limiting shall prevent excessive requests.
- Security headers shall be enabled and validated.
- Vulnerability testing shall not expose critical security flaws.

5.5 Reliability and Stability

- Services shall restart automatically in case of failure.
- Failure of one microservice shall not affect the overall system availability.
- System shall demonstrate stable operation during testing phase.
- Data backup and recovery mechanisms shall be verified.

5.6 User Acceptance Testing (UAT)

- End users (students, representatives, and administrators) shall test the system.
- No critical or major defects shall remain unresolved.
- All identified issues during testing shall be fixed before deployment.

---

06 . References

The following documents and standards were referred to during the preparation of this Software Requirements Specification (SRS):

1. IEEE Std 830-1984 – IEEE Recommended Practice for Software Requirements Specifications.

2. IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
3. Spring Boot Official Documentation – https://spring.io/projects/spring-boot
4. Spring Security Documentation – https://spring.io/projects/spring-security
5. Docker Documentation – https://docs.docker.com
6. Kubernetes Documentation – https://kubernetes.io/docs
7. Apache Kafka Documentation – https://kafka.apache.org/documentation
8. MinIO Object Storage Documentation – https://min.io/docs
9. OWASP Security Guidelines – https://owasp.org
10. REST API Design Guidelines – Industry best practices for RESTful API design.

Revision Notes / Written By

- Who prepared the document
- When it was written
- What changes were made
- Version history

Example

1.0	11 February 2026	Initial draft of SRS document	Pradeep kumar